LEZIONE 7 - Variabili e Tipi di Dato

Durata: 4 ore | Teoria: 2h | Esercizi: 2h

PARTE TEORICA (2 ore)

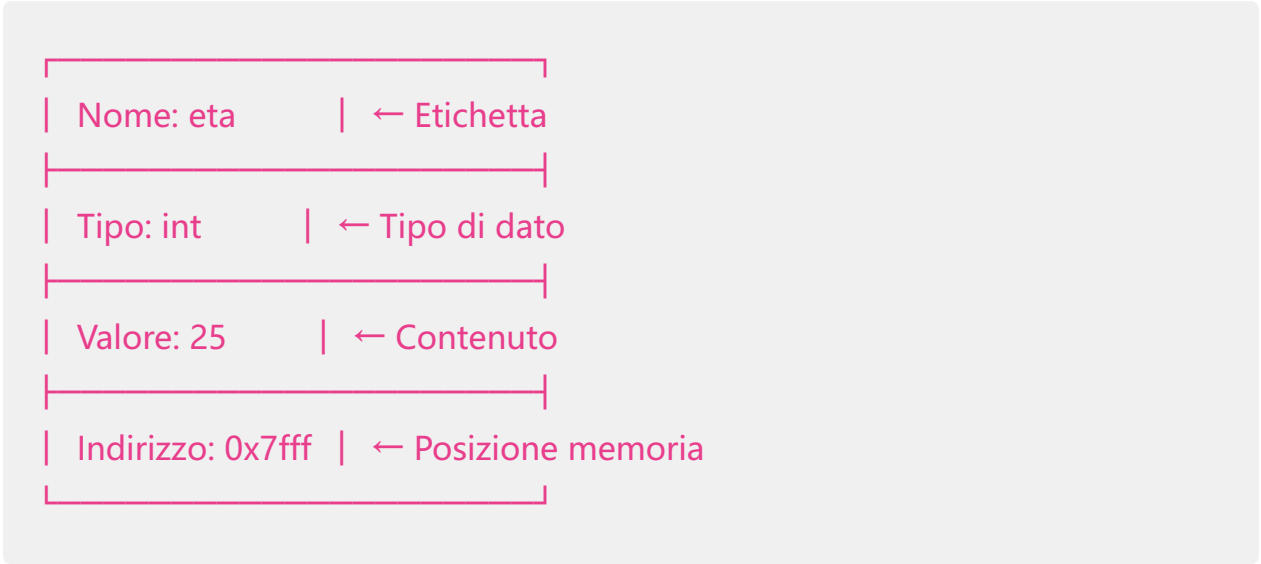
7.1 Concetto di Variabile (15 min)

Cos'è una Variabile?

Una **variabile** è un contenitore che può memorizzare un valore in memoria durante l'esecuzione del programma.

Analogia: Una variabile è come una scatola etichettata:

- **Nome** (etichetta): identifica la variabile
- **Tipo** (dimensione scatola): determina cosa può contenere
- **Valore** (contenuto): dato memorizzato
- **Indirizzo** (posizione): dove si trova in memoria



Caratteristiche Variabili in C

1. Tipizzazione Statica:

- Il tipo è dichiarato e non cambia
- Controllato a compile-time

```
int x = 5;    // x è intero e rimarrà intero
x = "ciao";   // ERRORE! Non puoi assegnare stringa
```

2. Dichiarazione Obbligatoria:

- Devi dichiarare prima di usare
- Il compilatore riserva spazio in memoria

```
int x;       // Dichiarazione
x = 10;      // Assegnazione

// OPPURE

int y = 20;   // Dichiarazione + Inizializzazione
```

3. Scope (Ambito):

- **Locali**: dentro funzioni, visibili solo lì
- **Globali**: fuori funzioni, visibili ovunque

```
int globale = 100;  // Variabile globale

int main() {
    int locale = 50;  // Variabile locale
    printf("%d\n", globale);  // OK
    printf("%d\n", locale);   // OK
    return 0;
}
```

```c
void altra_funzione() {
    printf("%d\n", globale);  // OK
    printf("%d\n", locale);   // ERRORE! locale non visibile qui
}
```

Memoria e Variabili

Quando dichiari una variabile, il compilatore:

1. **Alloca spazio** in memoria (RAM o registri)
2. **Assegna indirizzo** univoco
3. **Associa nome** all'indirizzo

Esempio in memoria:

```
Indirizzo | Variabile | Valore
----------|-----------|--------
0x1000    | int x     | 42
0x1004    | float y   | 3.14
0x1008    | char c    | 'A'
0x1009    | int z     | 100
```

7.2 Tipi di Dato Primitivi (25 min)

Il C offre diversi **tipi primitivi** (built-in) per memorizzare dati.

Numeri Interi

1. int (Integer)

```c
int eta = 25;
int anno = 2024;
int temperatura = -5;
```

- **Dimensione**: tipicamente 4 byte (32 bit)

- **Range**: -2,147,483,648 a +2,147,483,647
- **Uso**: numeri interi, contatori, indici

2. short (Short Integer)

```
short giorno = 15;
short mese = 10;
```

- **Dimensione**: tipicamente 2 byte (16 bit)
- **Range**: -32,768 a +32,767
- **Uso**: quando serve meno spazio

3. long (Long Integer)

```
long popolazione = 1400000000L;  // Nota la 'L'
long distanza = 384400000L;
```

- **Dimensione**: tipicamente 4 o 8 byte
- **Range**: almeno come int, spesso 64 bit
- **Uso**: numeri molto grandi

4. long long

```
long long big_number = 9223372036854775807LL;  // Nota 'LL'
```

- **Dimensione**: 8 byte (64 bit)
- **Range**: -9,223,372,036,854,775,808 a +9,223,372,036,854,775,807
- **Uso**: numeri enormi (timestamp, grandi calcoli)

5. Unsigned (senza segno)

```
unsigned int positivo = 42;
unsigned short porta = 8080;
unsigned long id = 4294967295UL;
```

- **Unsigned**: solo numeri positivi (no negativi)

- **Range raddoppia** in positivo
- **unsigned int**: 0 a 4,294,967,295

Tabella Riassuntiva Interi:

Tipo	Dimensione	Range (signed)	Range (unsigned)
char	1 byte	-128 a 127	0 a 255
short	2 byte	-32,768 a 32,767	0 a 65,535
int	4 byte	-2,147,483,648 a 2,147,483,647	0 a 4,294,967,295
long	4/8 byte	varia	varia
long long	8 byte	-9,223,372,036,854,775,808 a …	0 a 18,446,744,073,709,551,615

Numeri Decimali (Floating Point)

1. float (Single Precision)

```
float pi = 3.14159f;  // Nota la 'f'
float temperatura = 36.5f;
float peso = 75.3f;
```

- **Dimensione**: 4 byte (32 bit)
- **Precisione**: ~7 cifre decimali
- **Range**: ±3.4 × 10^{38}
- **Uso**: calcoli scientifici base, coordinate

2. double (Double Precision)

```
double pi_preciso = 3.141592653589793;
double distanza_terra_sole = 1.496e11;  // Notazione scientifica
```

- **Dimensione**: 8 byte (64 bit)

- **Precisione**: ~15 cifre decimali
- **Range**: ±1.7 × 10^{308}
- **Uso**: calcoli precisi, default per float in C

3. long double (Extended Precision)

```
long double molto_preciso = 3.14159265358979323846264338327 9L;
```

- **Dimensione**: 10, 12 o 16 byte (dipende da sistema)
- **Precisione**: ~19 cifre decimali
- **Uso**: calcoli estremamente precisi

Confronto float vs double:

```
float f = 0.1f + 0.2f;
double d = 0.1 + 0.2;

printf("float:  %.20f\n", f);   // 0.30000001192092895508
printf("double: %.20f\n", d);   // 0.29999999999999998890

// Nessuno è esattamente 0.3!
```

Caratteri

char (Character)

```
char lettera = 'A';
char cifra = '5';
char simbolo = '@';
char newline = '\n';
```

- **Dimensione**: 1 byte (8 bit)
- **Range**: -128 a 127 (signed) o 0 a 255 (unsigned)
- **Uso**: singolo carattere, piccoli interi
- **Importante**: `'A'` ≠ `"A"` (carattere vs stringa)

char come intero:

```
char c = 65;  // Equivalente a 'A' (codice ASCII)
printf("%c\n", c);  // Stampa: A
printf("%d\n", c);  // Stampa: 65
```

Tipo Booleano (C99+)

_Bool / bool

```
#include <stdbool.h>  // Per 'bool', 'true', 'false'

bool attivo = true;
bool completato = false;

if (attivo) {
    printf("Sistema attivo\n");
}
```

- **Dimensione**: 1 byte
- **Valori**: 0 (false) o 1 (true)
- **Prima di C99**: si usava int (0 = falso, diverso da 0 = vero)

Tipo Void

void

```
void funzione_senza_ritorno() {
    printf("Non ritorno nulla\n");
    // Nessun return
}

void *puntatore_generico;  // Puntatore a tipo sconosciuto
```

- **Non è un valore**: indica “assenza di tipo”

- **Uso**: funzioni senza return, puntatori generici

7.3 Dichiarazione e Inizializzazione (15 min)

Dichiarazione

Sintassi:

```
tipo nome_variabile;
```

Esempi:

```
int x;
float temperatura;
char iniziale;
double pi;
```

Dichiarazioni multiple:

```
int a, b, c;          // Tre variabili int
float x, y, z;        // Tre variabili float
int i = 0, j = 1, k;  // Due inizializzate, una no
```

Inizializzazione

Inizializzazione diretta:

```
int x = 10;
float y = 3.14f;
char c = 'A';
```

Inizializzazione vs Assegnazione:

```c
// Inizializzazione (alla dichiarazione)
int x = 5;

// Assegnazione (dopo dichiarazione)
int y;
y = 10;
```

Valori Non Inizializzati

ATTENZIONE: Variabili non inizializzate contengono **garbage** (spazzatura)!

```c
int x;  // NON inizializzato
printf("%d\n", x);  // Stampa valore casuale (garbage)!
```

Comportamento:

- **Variabili locali**: garbage (valori casuali in memoria)
- **Variabili globali/statiche**: inizializzate a 0 automaticamente

```c
int globale;  // Automaticamente = 0

int main() {
    int locale;  // Contiene garbage!

    printf("Globale: %d\n", globale);  // Stampa: 0
    printf("Locale: %d\n", locale);    // Stampa: ??? (casuale)

    return 0;
}
```

Best Practice: Inizializza SEMPRE le variabili!

```c
// ✓ BUONO
int x = 0;
```

```
float y = 0.0f;
char c = '\0';

// ✗ CATTIVO
int x;
float y;
char c;
```

Costanti

const (variabili immutabili):

```
const int MAX_STUDENTI = 100;
const float PI = 3.14159f;
const char NUOVA_LINEA = '\n';

// Tentativo di modifica → ERRORE
MAX_STUDENTI = 200;  // ERRORE! const non modificabile
```

#define (macro preprocessore):

```
#define MAX_SIZE 1000
#define PI 3.14159
#define NOME_PROGRAMMA "MioApp"

// Uso:
int array[MAX_SIZE];
float circonferenza = 2 * PI * raggio;
```

Differenza const vs #define:

```
// const: variabile vera con tipo e indirizzo
const int x = 10;
printf("%p\n", &x);  // OK, ha indirizzo
```

```
// #define: semplice sostituzione testuale
#define Y 10
printf("%p\n", &Y);  // ERRORE! Y è sostituito con 10
```

7.4 Operatore di Assegnazione (10 min)

Assegnazione Base

Sintassi:

```
variabile = valore;
```

Esempi:

```
int x;
x = 42;            // Assegna 42 a x

float y;
y = 3.14f;         // Assegna 3.14 a y

char c;
c = 'A';           // Assegna 'A' a c
```

Assegnazioni Multiple

Concatenazione:

```
int a, b, c;
a = b = c = 10;     // Tutti = 10 (da destra a sinistra)

// Equivalente a:
c = 10;
```

```
b = c;
a = b;
```

Swap (scambio) di variabili:

```
int x = 5, y = 10, temp;

// Scambio tradizionale (con variabile temporanea)
temp = x;    // temp = 5
x = y;       // x = 10
y = temp;    // y = 5

printf("x = %d, y = %d\n", x, y);  // x = 10, y = 5
```

L-value e R-value

L-value (left-value): può stare a sinistra di `=` (indirizzo modificabile)
R-value (right-value): può stare solo a destra di `=` (valore)

```
int x = 5;
int y = 10;

x = y;       // OK: x è l-value, y è r-value
y = x + 5;   // OK: y è l-value, x+5 è r-value

10 = x;      // ERRORE! 10 è r-value, non può stare a sinistra
x + y = 15;  // ERRORE! x+y è r-value
```

7.5 Input con scanf() (25 min)

La funzione **scanf()** legge input da tastiera (standard input).

Sintassi Base

```
scanf("formato", &variabile);
```

Nota importante: Il simbolo **&** (indirizzo di) è **obbligatorio**!

Leggere Diversi Tipi

Numeri interi:

```
int eta;
printf("Inserisci età: ");
scanf("%d", &eta);
printf("Hai %d anni\n", eta);
```

Numeri decimali:

```
float altezza;
printf("Inserisci altezza (m): ");
scanf("%f", &altezza);
printf("Altezza: %.2f metri\n", altezza);
```

Caratteri:

```
char iniziale;
printf("Inserisci iniziale: ");
scanf(" %c", &iniziale);  // Nota lo SPAZIO prima di %c
printf("Iniziale: %c\n", iniziale);
```

Stringhe (anticipazione):

```
char nome[50];
printf("Inserisci nome: ");
```

```
scanf("%s", nome);  // Nota: NO & per array/stringhe!
printf("Ciao, %s!\n", nome);
```

Specificatori scanf()

Tipo	Specificatore	Esempio
int	%d	scanf(“%d”, &x)
unsigned int	%u	scanf(“%u”, &x)
short	%hd	scanf(“%hd”, &x)
long	%ld	scanf(“%ld”, &x)
long long	%lld	scanf(“%lld”, &x)
float	%f	scanf(“%f”, &x)
double	%lf	scanf(“%lf”, &x)
long double	%Lf	scanf(“%Lf”, &x)
char	%c	scanf(“%c”, &x)
string	%s	scanf(“%s”, str)

ATTENZIONE:

- printf float: `%f`
- scanf float: `%f`
- printf double: `%f` (o %lf)
- scanf double: **`%lf`** (importante!)

Input Multiplo

Più variabili in una volta:

```
int giorno, mese, anno;
printf("Inserisci data (gg mm aaaa): ");
```

```
scanf("%d %d %d", &giorno, &mese, &anno);
printf("Data: %02d/%02d/%d\n", giorno, mese, anno);
```

Input:

```
15 10 2024
```

Output:

```
Data: 15/10/2024
```

Problemi Comuni con scanf()

Problema 1: Carattere dopo numero

```
int num;
char car;

scanf("%d", &num);   // Legge numero
scanf("%c", &car);   // Legge '\n' residuo! BUG!

// SOLUZIONE: spazio prima di %c
scanf(" %c", &car);  // Salta whitespace
```

Problema 2: Buffer non svuotato

```
char c1, c2;
scanf("%c", &c1);  // Inserisci 'A' + INVIO
scanf("%c", &c2);  // Legge '\n'! Non chiede input

// SOLUZIONE 1: spazio
scanf(" %c", &c2);
```

```c
// SOLUZIONE 2: svuota buffer
while (getchar() != '\n');  // Consuma fino a newline
```

Problema 3: Input non valido

```c
int x;
scanf("%d", &x);  // Utente inserisce "abc"
// scanf fallisce, x rimane non modificato, buffer corrotto

// SOLUZIONE: controlla valore di ritorno
if (scanf("%d", &x) != 1) {
    printf("Input non valido!\n");
    while (getchar() != '\n');  // Svuota buffer
}
```

Valore di Ritorno scanf()

scanf() ritorna il numero di elementi letti con successo.

```c
int x, y;
int letti = scanf("%d %d", &x, &y);

if (letti == 2) {
    printf("Entrambi letti: %d, %d\n", x, y);
} else if (letti == 1) {
    printf("Solo primo letto: %d\n", x);
} else {
    printf("Nessun numero letto!\n");
}
```

Alternative a scanf()

fgets() per stringhe (più sicuro):

```c
char nome[50];
printf("Nome: ");
fgets(nome, 50, stdin);  // Legge max 49 caratteri + '\0'
```

getchar() per singolo carattere:

```c
char c;
printf("Premi un tasto: ");
c = getchar();
printf("Hai premuto: %c\n", c);
```

7.6 Operatore sizeof (10 min)

L'operatore **sizeof** restituisce la dimensione in byte di un tipo o variabile.

Sintassi

```c
sizeof(tipo)
sizeof(variabile)
sizeof variabile  // Parentesi opzionali per variabili
```

Esempi

Dimensione tipi:

```c
printf("sizeof(char): %zu byte\n", sizeof(char));
printf("sizeof(int): %zu byte\n", sizeof(int));
printf("sizeof(float): %zu byte\n", sizeof(float));
printf("sizeof(double): %zu byte\n", sizeof(double));
printf("sizeof(long long): %zu byte\n", sizeof(long long));
```

Output tipico (x86-64):

```
sizeof(char): 1 byte
sizeof(int): 4 byte
sizeof(float): 4 byte
sizeof(double): 8 byte
sizeof(long long): 8 byte
```

Dimensione variabili:

```c
int x = 42;
float y = 3.14f;
double z = 2.718;

printf("x occupa %zu byte\n", sizeof(x));  // 4
printf("y occupa %zu byte\n", sizeof(y));  // 4
printf("z occupa %zu byte\n", sizeof(z));  // 8
```

Array (anticipazione):

```c
int array[10];
printf("Array occupa %zu byte\n", sizeof(array));  // 40 (10 × 4)
printf("Elementi: %zu\n", sizeof(array) / sizeof(array[0]));  // 10
```

Note Importante

`%zu` per sizeof:

```c
// ✓ CORRETTO
printf("%zu\n", sizeof(int));

// ✗ SBAGLIATO (può funzionare ma non portabile)
printf("%d\n", sizeof(int));
printf("%lu\n", sizeof(int));
```

sizeof è compile-time:

```
int size = sizeof(int);  // Calcolato a compile-time, non runtime
```

7.7 Conversioni di Tipo (15 min)

Le **conversioni** permettono di cambiare tipo a un dato.

Conversioni Implicite (Automatiche)

Promozione intera:

```
char c = 'A';      // 1 byte
int i = c;         // Promosso automaticamente a int
printf("%d\n", i); // 65
```

Promozione in espressioni:

```
int x = 10;
float y = 3.5f;
float risultato = x + y;  // x promosso a float, risultato float
```

Perdita di informazione:

```
float f = 3.14f;
int i = f;         // f troncato a 3 (perde decimali)
printf("%d\n", i); // 3
```

Conversioni Esplicite (Casting)

Sintassi:

```
(tipo) espressione
```

Esempi:

```
int x = 10;
int y = 3;
float risultato = (float)x / y;  // 10.0 / 3 = 3.333...

// Senza cast:
float risultato2 = x / y;  // 10 / 3 = 3 (divisione intera!)
```

Cast necessari:

```
// Divisione intera vs reale
int a = 7, b = 2;
printf("Senza cast: %d\n", a / b);      // 3
printf("Con cast: %.2f\n", (float)a / b); // 3.50

// Perdita precisione controllata
double pi = 3.141592653;
int pi_int = (int)pi;  // Esplicito: voglio troncare
printf("%d\n", pi_int); // 3
```

Regole Conversione Automatica

Gerarchia tipi (dal più piccolo al più grande):

```
char → short → int → long → long long → float → double → long double
```

In espressioni miste:

- Il tipo più piccolo viene promosso al più grande
- Il risultato ha il tipo più grande

```
int i = 10;
float f = 3.5f;
double d = 2.7;
```

```
auto result = i + f;  // int → float, result è float
auto result2 = f + d; // float → double, result2 è double
```

Divisione intera vs reale:

```
int a = 5, b = 2;

int r1 = a / b;          // 2 (divisione intera)
float r2 = a / b;        // 2.0 (divisione intera, poi assegnato a float)
float r3 = (float)a / b;  // 2.5 (a promosso, divisione reale)
float r4 = a / (float)b;  // 2.5 (b promosso, divisione reale)
float r5 = (float)(a / b); // 2.0 (divisione intera, poi cast)
```

Warning e Conversioni

Il compilatore avvisa per conversioni pericolose:

```
double d = 3.14159;
int i = d;  // Warning: possibile perdita dati

// Soluzione: cast esplicito (dice "lo so, lo voglio")
int i = (int)d;  // OK, nessun warning
```

PARTE PRATICA (2 ore)

ESERCIZI GUIDATI

Esercizio 1: Dichiarazione e Inizializzazione (20 min)

1.1 Dichiara variabili di vari tipi e stampale:

```c
#include <stdio.h>

int main() {
    // Dichiarazione e inizializzazione
    int eta = 25;
    float altezza = 1.75f;
    double peso = 70.5;
    char iniziale = 'M';

    // Stampa
    printf("Età: %d anni\n", eta);
    printf("Altezza: %.2f metri\n", altezza);
    printf("Peso: %.1f kg\n", peso);
    printf("Iniziale: %c\n", iniziale);

    return 0;
}
```

Output:

```
Età: 25 anni
Altezza: 1.75 metri
Peso: 70.5 kg
Iniziale: M
```

1.2 Esperimento con valori non inizializzati:

```c
#include <stdio.h>

int globale_non_init;  // Globale

int main() {
    int locale_non_init;  // Locale
```

```
    printf("Globale non inizializzata: %d\n", globale_non_init);
    printf("Locale non inizializzata: %d\n", locale_non_init);

    // Valori attesi:
    // Globale: 0 (inizializzata automaticamente)
    // Locale: ??? (garbage, valore casuale)

    return 0;
}
```

1.3 Uso di const:

```
#include <stdio.h>

int main() {
    const float PI = 3.14159f;
    const int GIORNI_SETTIMANA = 7;

    float raggio = 5.0f;
    float circonferenza = 2 * PI * raggio;

    printf("Circonferenza: %.2f\n", circonferenza);
    printf("Giorni in una settimana: %d\n", GIORNI_SETTIMANA);

    // PI = 3.0f;  // ERRORE! const non modificabile

    return 0;
}
```

1.4 Sizeof dei tipi:

```
#include <stdio.h>

int main() {
```

```
    printf("=== DIMENSIONI TIPI ===\n");
    printf("char:       %zu byte\n", sizeof(char));
    printf("short:      %zu byte\n", sizeof(short));
    printf("int:        %zu byte\n", sizeof(int));
    printf("long:       %zu byte\n", sizeof(long));
    printf("long long:  %zu byte\n", sizeof(long long));
    printf("float:      %zu byte\n", sizeof(float));
    printf("double:     %zu byte\n", sizeof(double));
    printf("long double: %zu byte\n", sizeof(long double));

    return 0;
}
```

Esercizio 2: Input con scanf() (25 min)

2.1 Input singolo intero:

```
#include <stdio.h>

int main() {
    int numero;

    printf("Inserisci un numero intero: ");
    scanf("%d", &numero);

    printf("Hai inserito: %d\n", numero);
    printf("Il doppio è: %d\n", numero * 2);

    return 0;
}
```

Esempio esecuzione:

```
Inserisci un numero intero: 15
Hai inserito: 15
Il doppio è: 30
```

2.2 Input di più valori:

```c
#include <stdio.h>

int main() {
    int giorno, mese, anno;

    printf("Inserisci data di nascita (gg mm aaaa): ");
    scanf("%d %d %d", &giorno, &mese, &anno);

    printf("Sei nato il %02d/%02d/%d\n", giorno, mese, anno);

    int eta_approssimativa = 2024 - anno;
    printf("Età approssimativa: %d anni\n", eta_approssimativa);

    return 0;
}
```

2.3 Input float e calcolo IMC:

```c
#include <stdio.h>

int main() {
    float peso, altezza, imc;

    printf("=== CALCOLO IMC ===\n");
    printf("Inserisci peso (kg): ");
    scanf("%f", &peso);

    printf("Inserisci altezza (m): ");
```

```
    scanf("%f", &altezza);

    // Calcolo IMC = peso / (altezza²)
    imc = peso / (altezza * altezza);

    printf("\n=== RISULTATO ===\n");
    printf("Peso: %.1f kg\n", peso);
    printf("Altezza: %.2f m\n", altezza);
    printf("IMC: %.1f\n", imc);

    printf("\nClassificazione:\n");
    if (imc < 18.5) {
        printf("Sottopeso\n");
    } else if (imc < 25) {
        printf("Normopeso\n");
    } else if (imc < 30) {
        printf("Sovrappeso\n");
    } else {
        printf("Obesità\n");
    }

    return 0;
}
```

Esempio esecuzione:

```
=== CALCOLO IMC ===
Inserisci peso (kg): 70
Inserisci altezza (m): 1.75

=== RISULTATO ===
Peso: 70.0 kg
Altezza: 1.75 m
IMC: 22.9
```

```
Classificazione:
Normopeso
```

2.4 Input carattere (con problema e soluzione):

```c
#include <stdio.h>

int main() {
    int numero;
    char lettera;

    printf("Inserisci un numero: ");
    scanf("%d", &numero);

    // PROBLEMA: scanf lascia '\n' nel buffer
    printf("Inserisci una lettera: ");
    scanf(" %c", &lettera);  // SOLUZIONE: spazio prima di %c

    printf("\nHai inserito:\n");
    printf("Numero: %d\n", numero);
    printf("Lettera: %c\n", lettera);

    return 0;
}
```

Esercizio 3: Conversioni di Tipo (20 min)

3.1 Divisione intera vs reale:

```c
#include <stdio.h>

int main() {
    int a = 7, b = 2;
```

```c
    printf("=== DIVISIONE ===\n");
    printf("a = %d, b = %d\n\n", a, b);

    // Divisione intera
    int ris_int = a / b;
    printf("Divisione intera: %d / %d = %d\n", a, b, ris_int);

    // Divisione reale (cast)
    float ris_float = (float)a / b;
    printf("Divisione reale: %.2f\n", ris_float);

    // Modulo (resto)
    int resto = a % b;
    printf("Resto: %d %% %d = %d\n", a, b, resto);

    return 0;
}
```

Output:

```
=== DIVISIONE ===
a = 7, b = 2

Divisione intera: 7 / 2 = 3
Divisione reale: 3.50
Resto: 7 % 2 = 1
```

3.2 Troncamento e arrotondamento:

```c
#include <stdio.h>
#include <math.h>

int main() {
```

```c
    double pi = 3.14159;

    printf("Valore originale: %.5f\n", pi);

    // Conversione diretta (troncamento)
    int pi_int = (int)pi;
    printf("Cast a int: %d\n", pi_int);

    // Arrotondamento
    int pi_round = (int)round(pi);
    printf("Arrotondato: %d\n", pi_round);

    // Floor (arrotonda per difetto)
    int pi_floor = (int)floor(pi);
    printf("Floor: %d\n", pi_floor);

    // Ceil (arrotonda per eccesso)
    int pi_ceil = (int)ceil(pi);
    printf("Ceil: %d\n", pi_ceil);

    return 0;
}
```

Output:

```
Valore originale: 3.14159
Cast a int: 3
Arrotondato: 3
Floor: 3
Ceil: 4
```

3.3 Overflow e underflow:

```c
#include <stdio.h>
#include <limits.h>

int main() {
    printf("=== LIMITI INT ===\n");
    printf("INT_MAX = %d\n", INT_MAX);
    printf("INT_MIN = %d\n", INT_MIN);

    // Overflow
    int max = INT_MAX;
    printf("\nOverflow:\n");
    printf("max = %d\n", max);
    printf("max + 1 = %d (overflow!)\n", max + 1);

    // Underflow
    int min = INT_MIN;
    printf("\nUnderflow:\n");
    printf("min = %d\n", min);
    printf("min - 1 = %d (underflow!)\n", min - 1);

    return 0;
}
```

Output:

```
=== LIMITI INT ===
INT_MAX = 2147483647
INT_MIN = -2147483648

Overflow:
max = 2147483647
max + 1 = -2147483648 (overflow!)

Underflow:
```

```
min = -2147483648
min - 1 = 2147483647 (underflow!)
```

3.4 Conversione carattere-numero:

```c
#include <stdio.h>

int main() {
    char cifra = '7';
    int numero = cifra - '0';

    printf("Carattere: '%c'\n", cifra);
    printf("Codice ASCII: %d\n", cifra);
    printf("Valore numerico: %d\n", numero);

    printf("\n");

    // Viceversa
    int val = 5;
    char car = val + '0';

    printf("Numero: %d\n", val);
    printf("Carattere: '%c'\n", car);
    printf("Codice ASCII: %d\n", car);

    return 0;
}
```

Output:

```
Carattere: '7'
Codice ASCII: 55
Valore numerico: 7
```

```
Numero: 5
Carattere: '5'
Codice ASCII: 53
```

Esercizio 4: Progetti Completi (45 min)

4.1 Calcolatrice interattiva:

```c
#include <stdio.h>

int main() {
    float num1, num2;
    char operatore;
    float risultato;

    printf("====== CALCOLATRICE ======\n");
    printf("Inserisci primo numero: ");
    scanf("%f", &num1);

    printf("Inserisci operatore (+, -, *, /): ");
    scanf(" %c", &operatore);

    printf("Inserisci secondo numero: ");
    scanf("%f", &num2);

    printf("\n====== CALCOLO ======\n");

    if (operatore == '+') {
        risultato = num1 + num2;
        printf("%.2f + %.2f = %.2f\n", num1, num2, risultato);
    } else if (operatore == '-') {
        risultato = num1 - num2;
        printf("%.2f - %.2f = %.2f\n", num1, num2, risultato);
    } else if (operatore == '*') {
```

```
        risultato = num1 * num2;
        printf("%.2f × %.2f = %.2f\n", num1, num2, risultato);
    } else if (operatore == '/') {
        if (num2 != 0) {
            risultato = num1 / num2;
            printf("%.2f ÷ %.2f = %.2f\n", num1, num2, risultato);
        } else {
            printf("ERRORE: Divisione per zero!\n");
        }
    } else {
        printf("ERRORE: Operatore non valido!\n");
    }

    return 0;
}
```

4.2 Convertitore temperature:

```
#include <stdio.h>

int main() {
    float temperatura;
    int scelta;
    float celsius, fahrenheit, kelvin;

    printf("====== CONVERTITORE TEMPERATURE ======\n");
    printf("1. Celsius → Fahrenheit e Kelvin\n");
    printf("2. Fahrenheit → Celsius e Kelvin\n");
    printf("3. Kelvin → Celsius e Fahrenheit\n");
    printf("\nScegli opzione (1-3): ");
    scanf("%d", &scelta);

    printf("Inserisci temperatura: ");
    scanf("%f", &temperatura);
```

```
    printf("\n====== RISULTATI ======\n");

    if (scelta == 1) {
        celsius = temperatura;
        fahrenheit = (celsius * 9.0f / 5.0f) + 32.0f;
        kelvin = celsius + 273.15f;

        printf("%.2f°C = %.2f°F = %.2fK\n", celsius, fahrenheit, kelvin);

    } else if (scelta == 2) {
        fahrenheit = temperatura;
        celsius = (fahrenheit - 32.0f) * 5.0f / 9.0f;
        kelvin = celsius + 273.15f;

        printf("%.2f°F = %.2f°C = %.2fK\n", fahrenheit, celsius, kelvin);

    } else if (scelta == 3) {
        kelvin = temperatura;
        celsius = kelvin - 273.15f;
        fahrenheit = (celsius * 9.0f / 5.0f) + 32.0f;

        printf("%.2fK = %.2f°C = %.2f°F\n", kelvin, celsius, fahrenheit);

    } else {
        printf("Opzione non valida!\n");
    }

    return 0;
}
```

4.3 Calcolo scontrino con sconto:

```c
#include <stdio.h>

int main() {
    float prezzo1, prezzo2, prezzo3;
    float subtotale, sconto_percentuale, sconto_importo, totale;

    printf("====== CALCOLO SCONTRINO ======\n");
    printf("Prezzo articolo 1: ");
    scanf("%f", &prezzo1);

    printf("Prezzo articolo 2: ");
    scanf("%f", &prezzo2);

    printf("Prezzo articolo 3: ");
    scanf("%f", &prezzo3);

    printf("Percentuale sconto (0-100): ");
    scanf("%f", &sconto_percentuale);

    // Calcoli
    subtotale = prezzo1 + prezzo2 + prezzo3;
    sconto_importo = subtotale * sconto_percentuale / 100.0f;
    totale = subtotale - sconto_importo;

    // Stampa scontrino
    printf("\n====== SCONTRINO ======\n");
    printf("Articolo 1:      %7.2f€\n", prezzo1);
    printf("Articolo 2:      %7.2f€\n", prezzo2);
    printf("Articolo 3:      %7.2f€\n", prezzo3);
    printf("----------------------\n");
    printf("Subtotale:       %7.2f€\n", subtotale);
    printf("Sconto (%.0f%%):   -%6.2f€\n", sconto_percentuale,
           sconto_importo);
    printf("========================\n");
```

```
    printf("TOTALE:         %7.2f€\n", totale);

    return 0;
}
```

Esempio output:

```
====== CALCOLO SCONTRINO ======
Prezzo articolo 1: 15.50
Prezzo articolo 2: 23.00
Prezzo articolo 3: 8.90
Percentuale sconto (0-100): 10

====== SCONTRINO ======
Articolo 1:        15.50€
Articolo 2:        23.00€
Articolo 3:         8.90€
----------------------
Subtotale:         47.40€
Sconto (10%):      -4.74€
========================
TOTALE:            42.66€
```

4.4 Conversione unità di misura:

```
#include <stdio.h>

int main() {
    float valore;
    int tipo;

    printf("====== CONVERTITORE UNITÀ ======\n");
    printf("1. Metri ↔ Piedi\n");
    printf("2. Chilogrammi ↔ Libbre\n");
```

```
    printf("3. Chilometri ↔ Miglia\n");
    printf("\nScegli conversione (1-3): ");
    scanf("%d", &tipo);

    printf("Inserisci valore: ");
    scanf("%f", &valore);

    printf("\n====== RISULTATO ======\n");

    if (tipo == 1) {
        float piedi = valore * 3.28084f;
        printf("%.2f metri = %.2f piedi\n", valore, piedi);

    } else if (tipo == 2) {
        float libbre = valore * 2.20462f;
        printf("%.2f kg = %.2f libbre\n", valore, libbre);

    } else if (tipo == 3) {
        float miglia = valore * 0.621371f;
        printf("%.2f km = %.2f miglia\n", valore, miglia);

    } else {
        printf("Opzione non valida!\n");
    }

    return 0;
}
```

4.5 Calcolo prestito bancario:

```
#include <stdio.h>
#include <math.h>

int main() {
```

```c
float capitale, tasso_annuo, tasso_mensile;
int durata_mesi;
float rata_mensile, totale_pagato, interessi_totali;

printf("====== CALCOLO RATA PRESTITO ======\n");
printf("Capitale richiesto (€): ");
scanf("%f", &capitale);

printf("Tasso interesse annuo (%%): ");
scanf("%f", &tasso_annuo);

printf("Durata (mesi): ");
scanf("%d", &durata_mesi);

// Calcolo rata (formula ammortamento francese)
tasso_mensile = tasso_annuo / 100.0f / 12.0f;

if (tasso_mensile > 0) {
    rata_mensile = capitale * tasso_mensile *
                   pow(1 + tasso_mensile, durata_mesi) /
                   (pow(1 + tasso_mensile, durata_mesi) - 1);
} else {
    rata_mensile = capitale / durata_mesi;
}

totale_pagato = rata_mensile * durata_mesi;
interessi_totali = totale_pagato - capitale;

printf("\n====== RIEPILOGO ======\n");
printf("Capitale:          %10.2f€\n", capitale);
printf("Tasso annuo:       %9.2f%%\n", tasso_annuo);
printf("Durata:            %10d mesi\n", durata_mesi);
printf("-----------------------------------\n");
printf("Rata mensile:      %10.2f€\n", rata_mensile);
printf("Totale da pagare:  %10.2f€\n", totale_pagato);
```

```
    printf("Interessi totali:    %10.2f€\n", interessi_totali);

    return 0;
}
```

ESERCIZI AUTONOMI

Esercizio 5: Pratica Individuale

5.1 Crea un programma che chiede base e altezza di un triangolo e calcola l'area.

5.2 Scrivi un programma che converte ore, minuti, secondi in secondi totali.

5.3 Crea un programma che calcola il costo di una chiamata telefonica (€/minuto × durata).

5.4 Programma che converte una distanza in metri in: km, cm, mm, miglia.

5.5 Calcolo consumo carburante: chiedi km percorsi e litri consumati, calcola km/l e l/100km.

ESERCIZI DA SVOLGERE A CASA

Homework 1: Scrivi un programma che:

- Chiede nome, età, altezza, peso
- Calcola IMC
- Calcola anno di nascita approssimativo
- Stampa un riepilogo formattato

Homework 2: Crea un convertitore valute che:

- Chiede importo in Euro
- Converte in: Dollari (1€ = 1.10$), Sterline (1€ = 0.85£), Yen (1€ = 160¥)
- Stampa tabella con tutte le conversioni

Homework 3: Programma calcolo stipendio:

- Chiede ore lavorate
- Chiede paga oraria
- Calcola stipendio lordo
- Calcola tasse (23% fino a 1000€, 30% oltre)
- Calcola stipendio netto
- Stampa dettaglio completo

Homework 4: Calcolatore geometria:

- Menu con 3 opzioni: cerchio, rettangolo, triangolo
- Per ogni figura: chiede dimensioni necessarie
- Calcola perimetro e area
- Stampa risultati formattati

Homework 5: Esperimenti con limiti numerici:

- Usa `<limits.h>` e `<float.h>`
- Stampa limiti di tutti i tipi interi
- Stampa limiti e precisione dei float
- Sperimenta con overflow/underflow
- Documenta risultati

Homework 6: Gioco “Indovina l’operazione”:

- Programma mostra: “5 ? 3 = 15”
- Utente deve indovinare l’operazione (risponde: *, +, -, /)
- Programma verifica e comunica se corretto
- Usa variabili per numeri e risultato

APPROFONDIMENTI

Rappresentazione Interna dei Tipi

Interi in memoria:

```
int x = 42;

Memoria (32 bit, little-endian):
Indirizzo: 0x1000 0x1001 0x1002 0x1003
Byte:      2A     00     00     00
           ↑      ↑      ↑      ↑
           LSB                  MSB

Binario: 00000000 00000000 00000000 00101010
```

Float in memoria (IEEE 754):

```
float f = 3.14f;

Binario: 01000000 01001000 11110101 11000011
         ↓      ↓        ↓         ↓
         Sign  Exponent    Mantissa
```

Allineamento Memoria (Alignment)

Le variabili sono **allineate** in memoria per efficienza.

```c
struct esempio {
    char c;    // 1 byte
    // 3 byte padding
    int i;     // 4 byte
    char c2;   // 1 byte
    // 3 byte padding
};

sizeof(struct esempio) = 12 byte (non 6!)
```

Padding inserito per allineare `int` a multiplo di 4.

Volatile e Register

volatile: Dice al compilatore di non ottimizzare.

```
volatile int sensor_value;
```

register: Suggerisce di tenere in registro CPU (raramente usato oggi).

```
register int i;
```

Tipi Fixed-Width (C99)

<stdint.h> definisce tipi con dimensione garantita:

```
#include <stdint.h>

int8_t   x;
int16_t  y;
int32_t  z;
int64_t  w;

uint8_t  a;
uint16_t b;
```

RIEPILOGO CONCETTI CHIAVE

Dichiarazione Variabili

```
tipo nome;
tipo nome = valore;
const tipo nome = valore;
```

Tipi Fondamentali

```
Interi:  char, short, int, long, long long
Float:   float, double, long double
Carattere: char
Booleano: _Bool / bool (C99+)
```

Input/Output

```
printf("formato", args);
scanf("formato", &var);
```

Conversioni

```
(tipo)espressione
tipo var = espressione;
```

Operatori Base

```
=   assegnazione
sizeof(tipo)  dimensione
```

TABELLE DI RIFERIMENTO RAPIDO

Tipi Interi (tipici)

Tipo	Byte	Range Signed	Range Unsigned
char	1	-128 a 127	0 a 255

Tipo	Byte	Range Signed	Range Unsigned
short	2	-32,768 a 32,767	0 a 65,535
int	4	-2,147,483,648 a 2,147,483,647	0 a 4,294,967,295
long	4/8	varia	varia
long long	8	-9.2×10^{18} a 9.2×10^{18}	0 a 1.8×10^{19}

Specificatori scanf

Tipo	Printf	Scanf
int	%d	%d
unsigned	%u	%u
long	%ld	%ld
float	%f	%f
double	%f o %lf	%lf
char	%c	%c
string	%s	%s

Header Files Utili

Header	Contenuto
<stdio.h>	I/O (printf, scanf)
<stdlib.h>	Utility (malloc, rand, exit)
<math.h>	Matematica (sqrt, pow, sin)
<string.h>	Stringhe (strlen, strcpy)

Header	Contenuto
<limits.h>	Limiti tipi interi
<float.h>	Limiti tipi float
<stdbool.h>	bool, true, false
<stdint.h>	Tipi fixed-width

PROBLEMI COMUNI E SOLUZIONI

Problema 1: Variabile non inizializzata

```
int x;
printf("%d\n", x);  // Garbage!
```

Soluzione: Inizializza sempre.

```
int x = 0;
```

Problema 2: scanf senza &

```
int x;
scanf("%d", x);  // ERRORE! Manca &
```

Soluzione:

```
scanf("%d", &x);
```

Problema 3: Float con scanf

```
double d;
scanf("%f", &d);  // ERRORE! double usa %lf
```

Soluzione:

```
scanf("%lf", &d);
```

Problema 4: Divisione intera

```
int a = 5, b = 2;
float r = a / b;  // r = 2.0 (non 2.5!)
```

Soluzione:

```
float r = (float)a / b;
```

Problema 5: Overflow silenzioso

```
int big = 2000000000;
int bigger = big + 1000000000;  // Overflow!
```

Soluzione: Usa tipo più grande.

```
long long bigger = (long long)big + 1000000000;
```

PUNTI CHIAVE DELLA LEZIONE

✓ **Variabile** = contenitore per dati in memoria
✓ **Tipi base**: int, float, double, char
✓ **Dichiarazione**: `tipo nome;` **Inizializzazione**: `tipo nome = valore;`
✓ **scanf()** legge input, richiede **&** prima della variabile
✓ **sizeof()** restituisce dimensione in byte
✓ **Cast** `(tipo)espressione` converte esplicitamente
✓ **Divisione intera**: `int / int = int` (tronca decimali)
✓ **const** rende variabile immutabile
✓ **Inizializza sempre** le variabili (no garbage)
✓ **%lf per double** in scanf (non %f)

PREPARAZIONE PER LA PROSSIMA LEZIONE

Nella prossima lezione studieremo:

- **Operatori aritmetici**: +, -, *, /, %
- **Operatori di incremento/decremento**: ++, –
- **Operatori relazionali**: <, >, ==, !=
- **Operatori logici**: &&, ||, !
- **Precedenza e associatività**
- **Espressioni complesse**

Porta con te:

- Esercizi homework completati
- Domande su scanf o conversioni
- Calcolatrice per verificare calcoli
- Esempi di espressioni matematiche complesse

Ottimo lavoro! Ora sai usare variabili e input! 🎯

Nelle prossime lezioni aggiungeremo operatori ed espressioni per creare calcoli complessi, poi passeremo ai costrutti di controllo (if, while, for) per programmi veramente potenti!